



Road Reorganization (roads)

O exército romano adquiriu recentemente novas carruagens, que podem transportar significativamente mais pessoas. Infelizmente, isso também significa que as carruagens são muito grandes e frequentemente ficam presas quando há tráfego na direção oposta. Para evitar isso, o imperador decidiu colocar com sentido único todas as estradas do império.

No entanto, o imperador gosta de visitar a sua cidade natal com frequência e quer ter a certeza de que, quando estiver lá, poderá regressar rapidamente a Roma se algo importante acontecer. Para garantir isso, ele quer ter a certeza de que o caminho mais curto da sua cidade natal até Roma não se tornará mais longo do que é atualmente. Além disso, ele também quer ter a certeza de que poderá regressar à sua cidade natal, mas isso não precisa ser o mais rápido possível.

O império romano contém N cidades e existem M estradas bidirecionais. A cidade natal do imperador é a cidade 0 e Roma é a cidade $N - 1$. Formalmente, ele quer escolher uma direção para cada estrada. Ele quer garantir que o caminho mais curto da cidade 0 à cidade $N - 1$ não seja mais longo do que era antes das estradas se tornarem estradas de sentido único. Além disso, ele quer garantir que ainda haja um caminho da cidade $N - 1$ de volta à cidade 0. A tua tarefa é orientar as estradas dessa forma ou informar ao imperador que isso é impossível.

⇒ É garantido que qualquer cidade do Império Romano é acessível a partir de qualquer outra cidade. Nota que isso não tem de ser o caso depois de orientar as estradas.

Nota também que orientar as estradas de forma que possas ir de 0 a $N - 1$ e voltar, sem necessariamente minimizar a distância de uma das viagens, ainda te dará pontos.

Implementação

Tens de submeter um único ficheiro de código .cpp.



Entre os ficheiros anexos deste problema encontra-se um template `roads.cpp` com um exemplo de implementação.

Tens de implementar a seguinte função:

C++

```
vector<pair<int, int>> orient_roads(int N, int M, vector<int> A,
vector<int> B, vector<int> W)
```

- O inteiro N representa o número de cidades.
- O inteiro M representa o número de estradas.
- Os arrays A , B e W descrevem as estradas do Império Romano: existe uma estrada entre a cidade $A[i]$ e a cidade $B[i]$ com comprimento $W[i]$.
- Se houver uma maneira de orientar as estradas na forma pedida, a função deve devolver um vetor de pares de comprimento M . Aqui, o i -ésimo par $\{F, T\}$ significa que há uma estrada orientada de F para T .
- Se não houver maneira de orientar as estradas, a função deve devolver um vetor vazio.

É garantido que existe, no máximo, uma estrada entre duas cidades e que, para todos os i , $A[i] \neq B[i]$. Se a função não devolver um vetor de comprimento 0 ou M , isso será considerado uma resposta errada. Todas as estradas mencionadas no input devem estar presentes no vetor de saída exatamente uma vez e nenhuma outra estrada deve estar presente. A ordem das estradas no vetor devolvido não importa.

Avaliador de Exemplo

Entre os ficheiros anexos deste problema, encontrarás uma versão simplificada do avaliador usado numa submissão real, que poderás usar para testar as soluções localmente. O avaliador lê dados de `stdin`, chama a função `orient_roads` e escreve de volta em `stdout` usando o seguinte formato.

O input é constituído por $M + 1$ linhas, contendo:

- Linha 1: os inteiros N e M .
- Linha $2 + i$ ($0 \leq i < M$): os inteiros A_i , B_i e W_i .

O output é constituído por 1 ou $M + 1$ linhas, contendo:

- Linha 1: **Yes** ou **No**, indicando se é possível orientar as estradas dessa forma.
- Se a resposta for **Yes**: Linha $2 + i$ ($0 \leq i < M$): F_i e T_i , indicando que a estrada entre estas duas cidades deve ser orientada de F_i para T_i .

No output, as orientações das estradas são impressas na ordem em que são devolvidas pela função.

Restrições

- $3 \leq N \leq 100\,000$.
- $N - 1 \leq M \leq 100\,000$.
- $0 \leq A_i, B_i \leq N - 1$ and $A_i \neq B_i$.
- $1 \leq W_i \leq 1\,000\,000\,000$.
- É garantido que qualquer cidade do império romano é atingível a partir de qualquer outra cidade.

Pontuação

O teu programa será testado com vários casos de teste agrupados em subtarefas. Para obter a pontuação de uma subtarefa, o teu programa tem de resolver corretamente todos os seus casos de teste. Se a solução for válida, mas não ótima, obterás metade dos pontos dessa subtarefa.

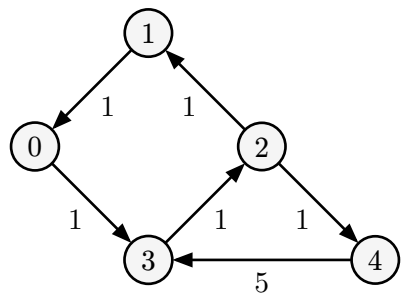
- **Subtarefa 0 [0 pontos]**: Casos de exemplo.
- **Subtarefa 1 [6 pontos]**: $\max(N, M) \leq 20$.
- **Subtarefa 2 [11 pontos]**: $M \leq N$.
- **Subtarefa 3 [17 pontos]**: É garantido que existe uma estrada direta entre a 0 e a cidade $N - 1$.
- **Subtarefa 4 [18 pontos]**: $\max(N, M) \leq 2000$.
- **Subtarefa 5 [17 pontos]**: $W_i = 1$.
- **Subtarefa 6 [31 pontos]**: Nenhuma restrição adicional.

⇒ Podes obter metade dos pontos de um caso de teste se o caminho mais curto da cidade 0 à $N - 1$ após as estradas terem sido direcionadas não tiver o mesmo comprimento que o caminho mais curto da cidade 0 à $N - 1$ antes das estradas terem sido direcionadas, mesmo que não houvesse maneira de orientar as estradas que satisfizesse plenamente o imperador.

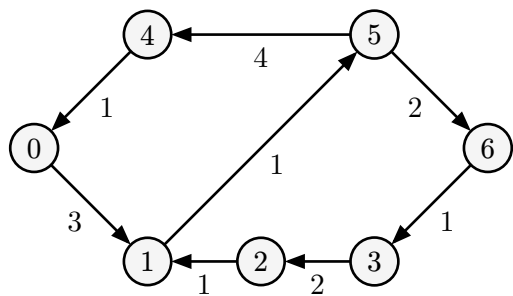
Exemplos

stdin	stdout
5 6 0 1 1 1 2 1 0 3 1 3 2 1 2 4 1 3 4 5	Yes 1 0 2 1 0 3 3 2 2 4 4 3
7 8 0 1 3 1 2 1 2 3 2 3 6 1 0 4 1 4 5 4 5 6 2 1 5 1	Yes 0 1 2 1 3 2 6 3 4 0 5 4 5 6 1 5
8 9 0 1 3 1 2 4 2 4 2 0 3 6 3 4 1 4 5 3 5 6 1 6 7 2 5 7 3	No

Explicação



No primeiro exemplo, podemos ir da cidade 0 para a 4 via $0 \rightarrow 3 \rightarrow 2 \rightarrow 4$. Isto é um caminho de comprimento 3, que é também o comprimento mínimo antes das estradas terem sido orientadas. Também podemos regressar via $4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$. Não importa o comprimento deste caminho.



No segundo exemplo, podemos ir de 0 para 6 usando a rota $0 \rightarrow 1 \rightarrow 5 \rightarrow 6$ e podemos regressar usando $6 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 5 \rightarrow 4 \rightarrow 0$.

No terceiro exemplo, pode ser mostrado que não existe nenhuma orientação de estradas que deixasse o imperador satisfeito.